

PROGRAM 1

Write a PL/SQL block to calculate the incentive of an employee whose ID is 110.

DECLARE

v_emp_id NUMBER := 110;

v_salary NUMBER;

v_incentive NUMBER

BEGIN

SELECT salary

INTO v_salary

FROM employees

WHERE employee_id = v_emp_id;

v_incentive := v_salary * 0.10;

DBMS_OUTPUT.PUT_LINE ('Employee ID: ' || v_emp_id);

DBMS_OUTPUT.PUT_LINE ('Salary: ' || v_salary);

DBMS_OUTPUT.PUT_LINE ('Incentive (10%): ' || v_incentive);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE ('No Employee found with ID ' ||
v_emp_id);

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE ('Error: ' || SQLERRM);

END;

/

PROGRAM 2

Write a PL/SQL block to show an invalid case-insensitive reference to a quoted and without quoted user-defined identifier.

DECLARE

"My Variable" NUMBER := 500;

BEGIN

DBMS_OUTPUT.PUT_LINE ("MyVariable");

END;

/

✓

PROGRAM 3

Write a PL/SQL block to adjust the salary of the employee whose ID 122.

Sample table: employees

DECLARE:

V_OLD_salary

V_NEW_salary

BEGIN

SELECT salary INTO V_OLD_salary
FROM employees

WHERE employee_id = 122;

V_NEW_salary := V_OLD_salary * 1.10;

UPDATE employees

SET salary = V_NEW_salary

WHERE employee_id = 122;

DBMS_OUTPUT.PUT_LINE ('Employee ID 122 salary updated from '
|| V_OLD_salary || ' to ' || V_NEW_salary);

COMMIT;

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE ('No Employee found with
ID 122');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE ('Error: ' || SQLERRM);

END;

PROGRAM 4

Write a PL/SQL block to create a procedure using the "IS [NOT] NULL Operator" and show AND operator returns TRUE if and only if both operands are TRUE.

DECLARE

v_num1 NUMBER := 10;

v_num2 NUMBER := NULL;

v_result BOOLEAN;

PROCEDURE check-null-and-logic IS

BEGIN

IF v_num1 IS NOT NULL THEN

DBMS_OUTPUT.PUT_LINE ('v_num1 is NOT NULL');

ELSE

DBMS_OUTPUT.PUT_LINE ('v_num1 is NULL');

END IF;

IF v_num2 IS NOT NULL THEN

DBMS_OUTPUT.PUT_LINE ('v_num2 is NULL');

ELSE

DBMS_OUTPUT.PUT_LINE ('v_num2 is NOT NULL');

END IF;

IF (v_num1 IS NOT NULL) AND (v_num1 > 5) THEN

DBMS_OUTPUT.PUT_LINE ('Both condition are TRUE → AND

ELSE

RETURNS TRUE');

DBMS_OUTPUT.PUT_LINE ('one or both condition are FALSE →

END IF;

AND RETURNS FALSE');

IF (v_num2 IS NOT NULL) AND (v_num1 > 5) THEN

DBMS_OUTPUT.PUT_LINE ('Both condition are TRUE → AND RETURNS TRUE');

ELSE

DBMS_OUTPUT.PUT_LINE ('one or both conditions are FALSE →

END IF;

AND RETURNS FALSE');

END;

PROGRAM 5

Write a PL/SQL block to describe the usage of LIKE operator including wildcard characters and escape character.

```
SET SERVEROUTPUT ON;
```

```
BEGIN:
```

```
IF 'Praween' LIKE 'pra%' THEN
```

```
    DBMS_OUTPUT.PUT_LINE ('LIKE with %: Name starts  
with pra');
```

```
END IF;
```

```
IF 'Raj' LIKE 'R-j' THEN
```

```
    DBMS_OUTPUT.PUT_LINE ('LIKE with -: second letter  
can be anything');
```

```
END IF;
```

```
IF 'A-B' LIKE 'A\B' ESCAPE '\' THEN
```

```
    DBMS_OUTPUT.PUT_LINE ('LIKE with ESCAPE: Matched the  
actual underscore "_");
```

```
END IF;
```

```
END;
```

```
/
```


PROGRAM 6

Write a PL/SQL program to arrange the number of two variable in such a way that the small number will store in num_small variable and large number will store in num_large variable.

SET SERVEROUTPUT ON;

DECLARE

num1 NUMBER := 25;

num2 NUMBER := 10;

num_small NUMBER;

num_large NUMBER;

BEGIN

IF num1 < num2 THEN

num_small := num1;

num_large := num2;

ELSE

num_small := num2;

num_large := num1;

END IF;

DBMS_OUTPUT.PUT_LINE ('smaller number = ' || num_small);

DBMS_OUTPUT.PUT_LINE ('larger number = ' || num_large);

END;

/



PROGRAM 7

Write a PL/SQL procedure to calculate the incentive on a target achieved and display the message either the record updated or not.

```
DECLARE
    emp_id NUMBER := 122;
    target NUMBER := 75000;
    incentive NUMBER;
    rows_updated NUMBER;
BEGIN
    IF target >= 10000 THEN
        incentive := 5000;
    ELSEIF target >= 50000 THEN
        incentive := 2000;
    ELSE
        incentive := 0;
    END IF;

    UPDATE employee
    SET incentive = incentive
    WHERE employee_id = emp_id;

    rows_updated := SQL%ROWCOUNT;
    IF rows_updated > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Record updated successfully.
                                incentive = ' || incentive);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No Record found for emp id = ' ||
                                emp_id);
    END IF;
END;
```


PROGRAM 8

Write a PL/SQL procedure to calculate incentive achieved according to the specific sale limit.

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE calc_incentive(
    p_emp_id IN NUMBER,
    p_sales IN NUMBER
) IS
    v_incentive NUMBER;
BEGIN
    IF p_sales >= 10000 THEN
        v_incentive := 5000;
    ELSEIF p_sales >= 50000 THEN
        v_incentive := 2000;
    ELSE
        v_incentive := 0;
    END IF;

    UPDATE Employee
    SET Incentive = v_incentive
    WHERE employee_id = p_emp_id;

    IF SQL%ROWCOUNT > 0 THEN
        DBMS_OUTPUT.PUT_LINE ('Record updated. Incentive = ' || v_incentive);
    ELSE
        DBMS_OUTPUT.PUT_LINE ('No record found for employee ' || p_emp_id);
    END IF;
END;

/
```

167

PROGRAM 9

Write a PL/SQL program to count number of employees in department 50 and check whether this department have any vacancies or not. There are 45 vacancies in this department.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_emp_count NUMBER;
```

```
    v_vacancies NUMBER=45;
```

```
    v_remaining NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT (*) INTO v_emp_count
```

```
    FROM employee
```

```
    WHERE department_id = 50;
```

```
    v_remaining := v_vacancies - v_emp_count;
```

```
    DBMS_OUTPUT.PUT_LINE ('Number of employees in Department 50' ||  
v_emp_count);
```

```
IF v_remaining > 0 THEN
```

```
    DBMS_OUTPUT.PUT_LINE ('Vacancies available : ' || v_remaining)
```

```
ELSE
```

```
    DBMS_OUTPUT.PUT_LINE ('No vacancies in Department 50:');
```

```
END IF;
```

```
END;
```

168

/

PROGRAM 10

Write a PL/SQL program to count number of employees in a specific department and check whether this department have any vacancies or not. If any vacancies, how many vacancies are in that department.

```
SET SERVEROUTPUT ON;

DECLARE
    v_dept_id NUMBER := & dept_id;
    v_emp_count NUMBER;
    v_total_posts NUMBER := 45;
    v_vacancies NUMBER;

BEGIN
    SELECT COUNT (*) INTO v_emp_count
    FROM employee
    WHERE department_id = v_dept_id;

    v_vacancies := v_total_posts - v_emp_count;

    DBMS_OUTPUT.PUT_LINE ('Department ID: ' || v_dept_id);
    DBMS_OUTPUT.PUT_LINE ('Number of Employees: ' || v_emp_count);
    IF v_vacancies > 0 THEN
        DBMS_OUTPUT.PUT_LINE ('Vacancies available: ' || v_vacancies);
    ELSE
        DBMS_OUTPUT.PUT_LINE ('NO vacancies in this department');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE ('INVALID DEPARTMENT ID');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE ('Error: ' || SQLERRM);
END;
```


PROGRAM 11

Write a PL/SQL program to display the employee IDs, names, job titles, hire dates, and salaries of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  v_emp_id  employee.employee-id % TYPE;
```

```
  v_name    employee.first-name % TYPE;
```

```
  v_job     employee.job-id % TYPE;
```

```
  v-hire-date employee.hire-date % TYPE;
```

```
  v-salary  employee.salary % TYPE;
```

```
CURSOR emp_cur IS
```

```
  SELECT employee-id, first-name, job-id, hire-date, salary
 FROM employee;
```

```
BEGIN
```

```
  DBMS_OUTPUT.PUT_LINE ('EMP ID NAME      JOB-ID HIRE-DATE
                          SALARY');
```

```
FOR emp_rec IN emp_cur LOOP
```

```
  DBMS_OUTPUT.PUT_LINE
```

```
    emp_rec.employee-id || ' ' ||
```

```
    RPAD (emp_rec.first-name, 12) || ' ' ||
```

```
    RPAD (emp_rec.job-id, 12) || ' ' ||
```

```
    TO_CHAR (emp_rec.hire-date, 'DD-MON-YYYY') || ' ' ||
```

```
    emp_rec.salary
```

```
  );
```

```
END LOOP;
```

```
END;
```


PROGRAM 12

Write a PL/SQL program to display the employee IDs, names, and department names of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    CURSOR emp_cur IS
```

```
        SELECT e.employee_id,
```

```
               e.first_name,
```

```
               d.department_name
```

```
        FROM employee e
```

```
        JOIN department d
```

```
        ON e.department_id = d.department_id;
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('EMP_ID NAME DEPARTMENT');
```

```
    DBMS_OUTPUT.PUT_LINE('-----');
```

```
    FOR emp_rec IN emp_cur LOOP
```

```
        DBMS_OUTPUT.PUT_LINE
```

```
            emp_rec.employee_id || ' ' ||
```

```
            RPAD(emp_rec.first_name, 12) || ' ' ||
```

```
            emp_rec.department_name
```

```
        );
```

```
    END LOOP;
```

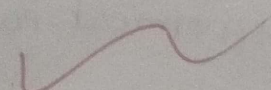
```
END;
```


PROGRAM 13

Write a PL/SQL program to display the job IDs, titles, and minimum salaries of all jobs.

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR job_cur IS
        SELECT job_id, job_title, min_salary
        FROM jobs;
BEGIN
    DBMS_OUTPUT.PUT_LINE ('JOB-ID   JOB-TITLE   MIN-SALARY');
    DBMS_OUTPUT.PUT_LINE ('-----');
    FOR job_rec IN job_cur LOOP
        DBMS_OUTPUT.PUT_LINE (
            RPAD (job_rec.job_id, 12) || ' ' ||
            RPAD (job_rec.job_title, 25) || ' ' ||
            job_rec.min_salary
        );
    END LOOP;
END;
```



PROGRAM 14

Write a PL/SQL program to display the employee IDs, names, and job history start dates of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    CURSOR emp_cur IS
```

```
        SELECT e.employee_id
```

```
              e.first_name,
```

```
              jh.start_date
```

```
        FROM employee e
```

```
        JOIN job_history jh
```

```
        ON e.employee_id = jh.employee_id;
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE ('EMP_ID  NAME  START_DATE');
```

```
    DBMS_OUTPUT.PUT_LINE ('-----');
```

```
    FOR emp_rec IN emp_cur LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            emp_rec.employee_id || ' ' ||
```

```
            RPAD (emp_rec.first_name, 12) || ' ' ||
```

```
            TO_CHAR (emp_rec.start_date, 'DD-MON-YYYY')
```

```
        );
```

```
    END LOOP;
```

```
END;
```

```
/
```


PROGRAM 15

Write a PL/SQL program to display the employee IDs, names, and job history end dates of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
CURSOR emp_wrt IS
```

```
SELECT e.employee_id,
```

```
e.first_name,
```

```
jh.end_date
```

```
FROM employee e
```

```
JOIN job_history jh
```

```
ON e.employee_id = jh.employee_id;
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE (
```

```
'EMP-ID NAME ENDDATE');
```

```
DBMS_OUTPUT.PUT_LINE (
```

```
'-----');
```

```
FOR emp_rec IN emp_wrt
```

```
LOOP
```

```
DBMS_OUTPUT.PUT_LINE (
```

```
emp_rec.employee_id || ' ' ||
```

```
RPAD (emp_rec.first_name, 12 || ' ' ||
```

```
TO_CHAR (emp_rec.end_date, 'DD-MON-YYYY');
```

```
);
```

```
END LOOP;
```

```
END;
```

```
/
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	